

A Practical Hierarchical Reinforcement Learning Framework for Slice-Aware Load Balancing and Mobility Control in O-RAN

Two-Agent Formulation with Continuous Local Control

Based on the Simplified HRL Design

Version 10.0 – June 4, 2026

Abstract

This document presents a two-agent hierarchical reinforcement learning (HRL) formulation for slice-aware load balancing and mobility control in O-RAN. The first agent is a global agent deployed as a Non-RT RIC rApp. It is the only learning component that observes network-wide slice load, SLA flags, and optionally connected-UE counters. It compresses this global information into per-gNB, per-slice coordination biases $b_{i,s} \in [-1, 1]$, indicating whether slice s at gNB i should offload users, remain neutral, or retain users. The second level consists of one local agent per gNB, deployed as near-RT RIC xApps. Each local agent runs at a faster timescale and deliberately does not observe PRB load values. Instead, it maps the global bias, connected-UE counters, SLA indicators, previous offsets, and mobility counters into neighbor-specific and slice-specific continuous A3 proto-offsets $\text{Offset}_{i,j,s} \in [-6, +6]$ dB. These proto-offsets are learned using TD3-style actor-critic reinforcement learning and then projected to the nearest valid discrete A3 offset before being sent to the gNB. This information-asymmetric design makes the global agent non-redundant: the global level decides why and when a gNB-slice should offload or retain users, while the local level decides how to execute this instruction safely through mobility parameters. The formulation does not optimize individual UE handovers. The gNB keeps executing the standard A3 handover rule; the learning system only adapts the slice-aware A3 offset parameters. Training follows a curriculum: static UEs with variable demand, static UEs with arrival/departure, slow mobility, and finally SUMO-based mobility. The training strategy uses one shared local TD3 policy that is first trained with rule-based global biases, then frozen and deployed as seven local executors while the global PPO agent is trained, and finally jointly fine-tuned with the global policy.

Contents

1	High-Level Idea	4
2	System Model	4
2.1	Network, Slices, and Neighbors	4

2.2	Slice Load	5
2.3	Connected UE Counter	5
2.4	Slice-Specific SLA Constraints	5
2.5	Mobility Counters	5
2.6	A3 Handover Model	6
3	Two-Agent HRL Formulation	6
3.1	Why Two Agents?	6
3.2	Timescales	6
4	Global Agent Formulation	7
4.1	Role	7
4.2	State	7
4.3	Action	7
4.4	Reward	7
4.5	Policy and Training	8
5	Local Agent Formulation	8
5.1	Role	8
5.2	Local State	8
5.3	Continuous Local Action	9
5.4	Projection to a Valid A3 Offset	9
5.5	Bias-to-Offset Tracking Function	9
5.6	Local Reward	10
5.7	Lagrange Multiplier Update	11
5.8	TD3 Actor-Critic Structure	11
6	Step-by-Step Study with Three gNBs	12
6.1	Initial Load Input	12
6.2	Global Agent Output	12
6.3	Local Agent 2 Observation	13
6.4	Local Actor Output and Quantization	13
6.5	A3 Calculation	13
6.6	Load and UE Counter Update	14
6.7	Local Reward Calculation	14
7	Training Workflow	15
7.1	Curriculum Training Scenarios	15
7.2	Stage A: Static UEs with Variable Demand	15
7.3	Stage B: Static UEs with Arrival and Departure	16
7.4	Stage C: Slow Mobility	16
7.5	Stage D: SUMO-Based Mobility	16
7.6	Complete HRL Training Pipeline	17
7.7	Training Hyperparameters	17
7.8	Why Poor Decisions Are Useful During Training	17

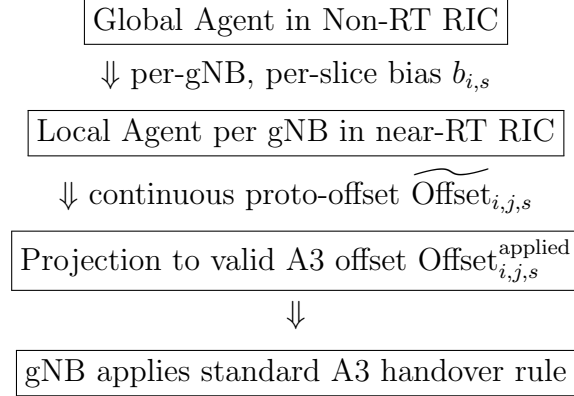
8	Training Strategy	17
8.1	Why Not Train Only One Lower Agent?	18
8.2	Information Split During Training	18
8.3	Phase 1: Pretrain the Shared Local TD3 Policy	19
8.4	Phase 2: Freeze the Local Policy and Train the Global PPO Agent . . .	19
8.5	Phase 3: Joint Fine-Tuning	19
8.6	Training Loop Summary	20
8.7	Baselines for Evaluation	20
9	Validation During Training	20
9.1	Simulator Sanity Validation	20
9.2	Validation of the Local TD3 Policy	21
9.3	Validation of the Global PPO Policy	22
9.4	Validation of the Complete HRL System	23
9.5	Validation Episodes Without Exploration Noise	23
9.6	Baseline-Based Validation	23
9.7	Practical Validation Checklist	24
10	Complexity Analysis	24
10.1	Global Complexity	24
10.2	Local Complexity	24
11	O-RAN Integration	25
11.1	Component Mapping	25
11.2	Example KPM Report	25
11.3	Example Control Message	25
12	Deployment Note	26
13	Implementation Notes	26
14	Conclusion	26
A	Acronyms	27

1 High-Level Idea

The objective is to balance slice load across neighboring gNBs without destroying mobility robustness. The key research idea is to use an information-asymmetric hierarchy: the global agent observes network-wide load and SLA conditions, while the local agents do not directly observe PRB load values. Instead, local agents receive a compressed global instruction $b_{i,s}$ and combine it with local counters and mobility KPIs. The problem is decomposed into two decisions:

1. **Strategic coordination:** identify which gNB-slice pairs should offload or retain traffic based on the global network state.
2. **Tactical mobility control:** decide the exact A3 offset to apply from each serving gNB to each target neighbor for each slice.

The architecture is therefore:



The local agent is not removed. It remains the main controller of the mobility parameter. However, its observation is intentionally restricted: it receives the global bias, UE counters, SLA indicators, previous offsets, and mobility counters, but not the direct PRB load values. This makes the global bias necessary rather than redundant. The only change from a discrete local DQN formulation is that the local policy now outputs continuous proto-offsets, which makes learning smoother. The final command remains compatible with the valid A3 offset set because the continuous output is quantized before application.

2 System Model

2.1 Network, Slices, and Neighbors

Definition 1 (gNB Set). Let $\mathcal{G} = \{1, 2, \dots, N\}$ denote the set of gNBs.

Definition 2 (Slice Set). Let $\mathcal{S} = \{\text{URLLC}, \text{eMBB}, \text{mMTC}\}$ denote the set of slices. The formulation also works for any number $S = |\mathcal{S}|$ of slices.

Definition 3 (Neighbor Set). For every serving gNB $i \in \mathcal{G}$, let $\mathcal{N}_i \subset \mathcal{G} \setminus \{i\}$ denote the set of candidate target neighbors. The average number of neighbors is denoted by $d = |\mathcal{N}_i|$.

2.2 Slice Load

For every gNB i and slice s , the normalized load is

$$L_{i,s}(t) = \frac{\text{PRB}_{\text{used},i,s}(t)}{\text{PRB}_{\text{total},i,s}(t)} \in [0, 1].$$

This is the main load-balancing measurement. If $L_{i,s}(t)$ is high, slice s at gNB i is consuming many radio resources.

2.3 Connected UE Counter

For every gNB and slice, the number of connected UEs is

$$K_{i,s}(t) = \sum_{u \in \mathcal{U}_i(t)} \mathbf{1}[\text{slice}(u) = s],$$

where $\mathcal{U}_i(t)$ is the set of UEs currently attached to gNB i . This counter is reported by the gNB as an aggregated KPI. The local agent does not need to know the identity or trajectory of every UE.

2.4 Slice-Specific SLA Constraints

Each slice has a different service requirement:

$$\begin{cases} D_{i,\text{URLLC}}(t) \leq D_{\max} = 10 \text{ ms}, \\ R_{i,\text{eMBB}}(t) \geq R_{\min} = 5 \text{ Mbps}, \\ Q_{i,\text{mMTC}}(t) \leq Q_{\max} = 1\%. \end{cases}$$

They are converted into non-positive constraint functions:

$$c_{i,\text{eMBB}}(t) = R_{\min} - R_{i,\text{eMBB}}(t), \quad (1)$$

$$c_{i,\text{URLLC}}(t) = D_{i,\text{URLLC}}(t) - D_{\max}, \quad (2)$$

$$c_{i,\text{mMTC}}(t) = Q_{i,\text{mMTC}}(t) - Q_{\max}. \quad (3)$$

A violation occurs when $c_{i,s}(t) > 0$, and the violation flag is

$$v_{i,s}(t) = \mathbf{1}[c_{i,s}(t) > 0].$$

2.5 Mobility Counters

The local agent observes handover behavior through aggregated counters, not through direct UE-level control. For each serving gNB i , target neighbor j , and slice s , define:

- $N_{i,j,s}^{\text{HOA}}$: number of handover attempts,
- $N_{i,j,s}^{\text{HOS}}$: number of successful handovers,
- $N_{i,j,s}^{\text{HOF}}$: number of handover failures,
- $N_{i,j,s}^{\text{PP}}$: number of ping-pong handovers.

The mobility ratios are

$$R_{i,j,s}^{\text{HF}}(t) = \frac{N_{i,j,s}^{\text{HOF}}(t)}{N_{i,j,s}^{\text{HOA}}(t) + \epsilon},$$

$$R_{i,j,s}^{\text{PP}}(t) = \frac{N_{i,j,s}^{\text{PP}}(t)}{N_{i,j,s}^{\text{HOS}}(t) + \epsilon},$$

where $\epsilon > 0$ avoids division by zero. These ratios allow the local agent to learn whether offloading toward a neighbor is stable or harmful.

2.6 A3 Handover Model

The simplified A3 condition is written using RSRP:

$$\text{RSRP}_j(t) > \text{RSRP}_i(t) + \text{Offset}_{i,j,s}^{\text{applied}}(t),$$

where i is the serving gNB, j is the target neighbor, and s is the slice. A negative offset makes handover easier, while a positive offset makes handover harder.

The valid A3 offset set is

$$\mathcal{A}_{\text{offset}} = \{-6, -4, -2, 0, +2, +4, +6\} \text{ dB}.$$

Although the local actor outputs a continuous proto-offset, the final applied value must belong to this set.

3 Two-Agent HRL Formulation

3.1 Why Two Agents?

A single centralized agent would have to choose offsets for all serving gNBs, all neighbors, and all slices. That action space grows quickly and is difficult to deploy in near-real-time. A fully distributed design is also weak because each gNB may offload traffic without knowing the global demand situation.

The proposed design uses two roles:

- **Global agent:** sees the full network state and coordinates demand direction.
- **Local agents:** one local agent per gNB, each using local and neighbor measurements to choose practical mobility parameters.

3.2 Timescales

Agent	Location	Period
Global agent	Non-RT RIC rApp	$T_H = 5 \text{ s}$
Local agent i	near-RT RIC xApp for gNB i	$T_L = 500 \text{ ms}$

The global agent updates slowly because it coordinates global load. The local agent updates faster because handover parameters must react to short-term SLA and mobility changes.

4 Global Agent Formulation

4.1 Role

The global agent answers:

Which slice at which gNB should offload, remain neutral, or retain users?

It does not directly configure A3 offsets. It produces a bias that the local agents use as high-level guidance.

4.2 State

The global state is the concatenation of load and SLA flags:

$$\mathbf{s}_H(t) = [L_{i,s}(t)]_{i \in \mathcal{G}, s \in \mathcal{S}} \parallel [v_{i,s}(t)]_{i \in \mathcal{G}, s \in \mathcal{S}}.$$

Thus,

$$\dim(\mathbf{s}_H) = 2NS.$$

Optionally, the connected UE counters can be included:

$$\mathbf{s}_H(t) = [L_{i,s}] \parallel [v_{i,s}] \parallel [K_{i,s}], \quad \dim(\mathbf{s}_H) = 3NS.$$

4.3 Action

The global action is a bias matrix:

$$\mathbf{a}_H(t) = \mathbf{b}(t) = [b_{i,s}(t)]_{i \in \mathcal{G}, s \in \mathcal{S}} \in [-1, 1]^{NS}.$$

The interpretation is:

- $b_{i,s} < 0$: slice s at gNB i should offload traffic.
- $b_{i,s} = 0$: neutral; local agent decides using local information.
- $b_{i,s} > 0$: slice s at gNB i should retain traffic.

4.4 Reward

The global reward promotes regional load balance:

$$r_H(t) = - \sum_{i=1}^N \sum_{s \in \mathcal{S}} \lambda_{\text{load}} (L_{i,s}(t) - L_{\text{target}})^2.$$

A light global SLA term may be added to discourage global actions that create many violations:

$$r_H(t) = - \sum_{i,s} \lambda_{\text{load}} (L_{i,s}(t) - L_{\text{target}})^2 - \rho_H \sum_{i,s} v_{i,s}(t).$$

The detailed SLA adaptation is handled by the local agents through Lagrangian penalties.

Inserted clarification after Section 4.3: Slice-aware global action space

Purpose of this clarification. The global action must not be interpreted as a single scalar bias per gNB. A scalar bias b_i would send the same high-level command to every slice at gNB i , which is not slice-aware. The correct action is a per-gNB, per-slice bias matrix

$$a_H(t) = B(t) = [b_{i,s}(t)]_{i \in \mathcal{G}, s \in \mathcal{S}} \in [-1, 1]^{N|\mathcal{S}|}. \quad (1)$$

Each entry controls one specific gNB-slice pair. For slice s at gNB i ,

- $b_{i,s} < 0$ means that slice s at gNB i should be encouraged to offload;
- $b_{i,s} \approx 0$ means neutral behavior;
- $b_{i,s} > 0$ means that slice s at gNB i should be retained.

Why one scalar per gNB is insufficient. If the global agent outputs only one value b_i , then all slices receive the same instruction:

$$b_i \rightarrow \{b_{i,\text{eMBB}}, b_{i,\text{URLLC}}, b_{i,\text{mMTC}}\} \quad \text{with all entries equal.} \quad (2)$$

This prevents the global controller from expressing situations such as

$$b_{i,\text{eMBB}} = -1, \quad b_{i,\text{URLLC}} = +1, \quad b_{i,\text{mMTC}} = 0, \quad (3)$$

which means: offload eMBB users from gNB i , retain URLLC users at gNB i , and keep mMTC neutral. Such a mixed instruction is essential because different slices can have different load and SLA conditions at the same gNB.

Action dimension. For N gNBs and $S = |\mathcal{S}|$ slices, the global PPO action dimension is

$$\dim(a_H) = NS. \quad (4)$$

For the seven-gNB, three-slice case used in this work,

$$\dim(a_H) = 7 \times 3 = 21. \quad (5)$$

An explicit ordering can be

$$a_H(t) = [b_{1,\text{eMBB}}, b_{1,\text{URLLC}}, b_{1,\text{mMTC}}, b_{2,\text{eMBB}}, b_{2,\text{URLLC}}, b_{2,\text{mMTC}}, \dots, b_{7,\text{eMBB}}, b_{7,\text{URLLC}}, b_{7,\text{mMTC}}]. \quad (6)$$

Compatibility with the local TD3 action. The local TD3 executor at gNB i receives the slice-specific bias $b_{i,s}$ in its local observation block and outputs neighbor-specific and slice-specific A3 proto-offsets

$$\tilde{a}_{L,i}(t) = [\widehat{\text{Offset}}_{i,j,s}(t) : j \in \mathcal{N}_i, s \in \mathcal{S}] \in [-6, +6]^{d_i S}. \quad (7)$$

Thus, the hierarchy remains slice-aware at both levels:

$$\text{Global PPO: } b_{i,s} \implies \text{Local TD3: } \text{Offset}_{i,j,s}^{\text{applied}}. \quad (8)$$

The global agent decides which gNB-slice pairs should offload, stay neutral, or retain traffic. The local agent decides how to realize that instruction safely for each neighbor and slice through valid A3 offsets.

Implementation note. The implementation should therefore expose the global action as a flattened vector of length NS , not length N . During execution, this vector is reshaped into a matrix $B \in \mathbb{R}^{N \times S}$ and each local executor i receives the row

$$B_i = [b_{i,\text{eMBB}}, b_{i,\text{URLLC}}, b_{i,\text{mMTC}}]. \quad (9)$$

This matches the local observation design and avoids collapsing slice-specific decisions into a single non-slice-aware command.

4.5 Policy and Training

The global policy is

$$\pi_H : \mathbb{R}^{2NS} \rightarrow [-1, 1]^{NS}.$$

A typical neural network is

$$\text{Input}(2NS) \rightarrow \text{FC}(256) \rightarrow \text{ReLU} \rightarrow \text{FC}(128) \rightarrow \text{ReLU} \rightarrow \text{FC}(NS) \rightarrow \tanh.$$

PPO is used because the global action is continuous and PPO is stable for slow-timescale policy learning.

Algorithm 1 Global PPO Training

- 1: Initialize global policy $\pi_{H,\theta}$ and value network V_ϕ .
 - 2: **for** each global training iteration **do**
 - 3: Observe $\mathbf{s}_H(t)$.
 - 4: Output $\mathbf{b}(t) = \pi_{H,\theta}(\mathbf{s}_H(t))$.
 - 5: Send $b_{i,s}(t)$ to the corresponding local agents.
 - 6: Simulate T_H seconds while local agents execute fast control steps.
 - 7: Compute $r_H(t)$.
 - 8: Update $\pi_{H,\theta}$ using PPO with generalized advantage estimation.
 - 9: **end for**
-

5 Local Agent Formulation

5.1 Role

There is one local agent per gNB. Local agent i answers:

Given the global bias and local mobility/SLA counters, what A3 offset should gNB i apply toward each neighbor j for each slice s ?

The local agent is responsible for fast, mobility-aware, slice-specific control.

5.2 Local State

For each neighbor $j \in \mathcal{N}_i$ and slice $s \in \mathcal{S}$, the local agent uses an information-restricted observation block:

$$\mathbf{o}_{i,j,s}(t) = \left[b_{i,s}, K_{i,s}, v_{i,s}, K_{j,s}, \text{Offset}_{i,j,s}^{\text{prev}}, R_{i,j,s}^{\text{HF}}, R_{i,j,s}^{\text{PP}} \right].$$

The PRB load values $L_{i,s}$ and $L_{j,s}$ are deliberately excluded from the local observation. They are visible to the global agent only, which transforms the global load/SLA situation into the bias $b_{i,s}$. This makes the global agent useful and prevents the local controller from becoming a second load-balancing agent.

The connected-UE counters are retained because they are lightweight aggregated counters and help the local agent distinguish between lightly and heavily populated neighbor-slice pairs without exposing the full PRB load state. The mobility ratios remain local because the local xApp must learn whether a specific neighbor direction is handover-safe.

The full local state is

$$\mathbf{o}_i(t) = \bigoplus_{s \in \mathcal{S}} \bigoplus_{j \in \mathcal{N}_i} \mathbf{o}_{i,j,s}(t).$$

The dimension is

$$\dim(\mathbf{o}_i) = 7Sd.$$

For $S = 3$ and $d = 2$, $\dim(\mathbf{o}_i) = 42$. For $S = 3$ and $d = 5$, $\dim(\mathbf{o}_i) = 105$.

5.3 Continuous Local Action

The local actor outputs one continuous proto-offset for every neighbor-slice pair:

$$\tilde{\mathbf{a}}_{L,i}(t) = \left[\widetilde{\text{Offset}_{i,j,s}(t)} : j \in \mathcal{N}_i, s \in \mathcal{S} \right] \in [-6, +6]^{Sd}.$$

This continuous action is useful for learning because the policy can change smoothly. For example, it can move from -4.8 to -4.3 to -3.7 instead of jumping only between discrete levels.

5.4 Projection to a Valid A3 Offset

The gNB receives only a valid offset from $\mathcal{A}_{\text{offset}}$. Therefore, every proto-offset is quantized:

$$\text{Offset}_{i,j,s}^{\text{applied}}(t) = Q \left(\widetilde{\text{Offset}_{i,j,s}(t)} \right),$$

where

$$Q(x) = \arg \min_{a \in \mathcal{A}_{\text{offset}}} |x - a|.$$

Examples:

$$Q(-4.6) = -4, \quad Q(3.7) = +4, \quad Q(4.5) = +4 \text{ or } +6$$

depending on the selected tie-breaking rule. In this document, ties are rounded to the nearest value with smaller magnitude to avoid unnecessary aggressive offset changes.

5.5 Bias-to-Offset Tracking Function

The global bias is strategic and not neighbor-specific. Since the local agent does not observe PRB load values, it interprets the bias together with a lightweight neighbor UE-pressure indicator. Define

$$\kappa_{j,s}(t) = \frac{K_{j,s}(t)}{K_s^{\max} + \epsilon} \in [0, +\infty),$$

where $K_s^{\max}$ is a reference maximum number of UEs for slice s and $\epsilon > 0$ avoids division by zero. The desired proto-offset is

$$\psi(b_{i,s}, \kappa_{j,s}) = \text{clip}_{[-6,6]} (6b_{i,s} + \alpha_K(\kappa_{j,s} - \kappa_{\text{target},s})).$$

Interpretation:

- If $b_{i,s} < 0$, the desired offset decreases, encouraging offloading from gNB i for slice s .

- If neighbor j has high UE pressure, $\kappa_{j,s} > \kappa_{\text{target},s}$, the desired offset increases, discouraging handover toward a crowded neighbor.
- If neighbor j has low UE pressure, $\kappa_{j,s} < \kappa_{\text{target},s}$, the desired offset decreases, making it easier to use that neighbor.

This function keeps the local decision neighbor-specific while preserving the role of the global agent as the only component with direct network-wide load visibility.

5.6 Local Reward

The local reward focuses on executing the global instruction safely. Direct load-balancing terms based on $L_{i,s}$ or $L_{j,s}$ are removed from the local reward to keep load optimization at the global level. The local reward combines bias tracking, SLA penalties, and mobility penalties:

$$\begin{aligned} \tilde{r}_{L,i}(t) = & -\lambda_{\text{track}} \sum_{s \in \mathcal{S}} \sum_{j \in \mathcal{N}_i} \left(\widetilde{\text{Offset}}_{i,j,s}(t) - \psi(b_{i,s}(t), \kappa_{j,s}(t)) \right)^2 \\ & - \sum_{s \in \mathcal{S}} \lambda_{i,s}(t) \max(0, c_{i,s}(t)) \\ & - \sum_{s \in \mathcal{S}} \sum_{j \in \mathcal{N}_i} (w_{\text{HF}} R_{i,j,s}^{\text{HF}}(t) + w_{\text{PP}} R_{i,j,s}^{\text{PP}}(t)) \\ & - \lambda_{\Delta} \sum_{s \in \mathcal{S}} \sum_{j \in \mathcal{N}_i} \left(\widetilde{\text{Offset}}_{i,j,s}(t) - \text{Offset}_{i,j,s}^{\text{prev}}(t) \right)^2. \end{aligned}$$

Default values can be

$$\lambda_{\text{track}} = 0.5, \quad w_{\text{HF}} = 1.0, \quad w_{\text{PP}} = 0.3, \quad \lambda_{\Delta} = 0.05.$$

The reward teaches the local agent that following the global offload/retain instruction is not sufficient. If an aggressive offset creates handover failures, ping-pong handovers, SLA violations, or unstable offset oscillations, the local reward becomes worse. The global reward remains responsible for measuring whether the overall slice-load distribution is improved.

Neighbor-pressure correction and local-global alignment. A possible local-global misalignment occurs if the global bias requests offloading toward a direction that is mobility-stable but leads to a crowded neighbor. The tracking target reduces this risk by including the neighbor-pressure term $\alpha_K(\kappa_{j,s} - \kappa_{\text{target},s})$. Therefore, even when $b_{i,s} < 0$ encourages offloading from serving gNB i , a neighbor with high connected-UE pressure $\kappa_{j,s} > \kappa_{\text{target},s}$ increases $\psi(b_{i,s}, \kappa_{j,s})$, which pushes the desired proto-offset upward and makes handover toward that neighbor harder. Conversely, a lightly populated neighbor with $\kappa_{j,s} < \kappa_{\text{target},s}$ decreases the desired proto-offset and becomes a more attractive offloading target.

This correction does not turn the local TD3 agent into a second load-balancing agent because it still does not observe the direct PRB load values $L_{i,s}$ or $L_{j,s}$. The connected-UE counter is used only as a lightweight tactical pressure signal. The global PPO agent remains responsible for deciding whether offloading or retention is globally beneficial based on network-wide load and SLA observations, while the local TD3 agent uses neighbor pressure, SLA penalties, and mobility penalties to execute that global instruction safely.

In this sense, the local reward does not eliminate all local-global tension; it bounds unsafe local execution and helps the global agent learn biases that are feasible for the local controllers to realize.

5.7 Lagrange Multiplier Update

The Lagrange multiplier for each gNB-slice pair is updated using the average constraint violation:

$$\lambda_{i,s} \leftarrow \max(0, \lambda_{i,s} + \eta_\lambda \bar{c}_{i,s}),$$

where

$$\bar{c}_{i,s} = \frac{1}{T_{\text{ep}}} \sum_t \max(0, c_{i,s}(t)).$$

If a slice violates its SLA frequently, $\lambda_{i,s}$ increases, so the corresponding penalty becomes stronger. If the SLA is satisfied, the multiplier remains small.

5.8 TD3 Actor-Critic Structure

The local agent uses TD3 because the local action is continuous. Each local agent i contains:

- an **actor** $\mu_{\theta_i}(\mathbf{o}_i)$ that outputs proto-offsets $\widetilde{\text{Offset}}_{i,j,s}$;
- two **critics** $Q_{\phi_{i,1}}(\mathbf{o}_i, \tilde{\mathbf{a}}_{L,i})$ and $Q_{\phi_{i,2}}(\mathbf{o}_i, \tilde{\mathbf{a}}_{L,i})$ that estimate the value of the action.

The two critics reduce overestimation by using the minimum target value.

The actor network can be written as

$$\mathbf{o}_i \rightarrow \text{FC}(256) \rightarrow \text{ReLU} \rightarrow \text{FC}(128) \rightarrow \text{ReLU} \rightarrow \text{FC}(Sd) \rightarrow 6 \tanh(\cdot).$$

The output dimension is Sd , one proto-offset per neighbor-slice pair.

Algorithm 2 Local Agent i Training with TD3 and Lagrangian Reward

- 1: Initialize actor μ_{θ_i} , critics $Q_{\phi_{i,1}}, Q_{\phi_{i,2}}$, target networks, replay buffer $\mathcal{D}_i$, and multipliers $\lambda_{i,s}$.
 - 2: **for** each local step **do**
 - 3: Observe local state $\mathbf{o}_i(t)$.
 - 4: Actor outputs continuous proto-action $\tilde{\mathbf{a}}_{L,i}(t) = \mu_{\theta_i}(\mathbf{o}_i(t)) + \epsilon_t$.
 - 5: Quantize each proto-offset to obtain $\widetilde{\text{Offset}}_{i,j,s}^{\text{applied}}(t) = Q(\widetilde{\text{Offset}}_{i,j,s}(t))$.
 - 6: Apply the quantized offsets in the simulator or gNB A3 model.
 - 7: Observe next state $\mathbf{o}_i(t+1)$, SLA values, mobility counters, and reward $\tilde{r}_{L,i}(t)$.
 - 8: Store $(\mathbf{o}_i(t), \tilde{\mathbf{a}}_{L,i}(t), \tilde{r}_{L,i}(t), \mathbf{o}_i(t+1))$ in $\mathcal{D}_i$.
 - 9: Sample a mini-batch from $\mathcal{D}_i$ and update the two critics.
 - 10: Every d_{policy} critic updates, update the actor using the critic gradient.
 - 11: Soft-update target networks.
 - 12: **end for**
 - 13: At the end of each episode, update $\lambda_{i,s}$ using average constraint violations.
-

6 Step-by-Step Study with Three gNBs

6.1 Initial Load Input

Consider three gNBs and three slices:

$$\mathcal{G} = \{gNB_1, gNB_2, gNB_3\}, \quad \mathcal{S} = \{\text{URLLC}, \text{eMBB}, \text{mMTC}\}.$$

The reported load matrix is

$$L(t) = \begin{bmatrix} 0.55 & 0.60 & 0.50 \\ 0.57 & 0.92 & 0.54 \\ 0.53 & 0.45 & 0.51 \end{bmatrix},$$

where columns correspond to URLLC, eMBB, and mMTC. Thus, gNB_2 is overloaded for eMBB:

$$L_{2,\text{eMBB}} = 0.92.$$

The target load is

$$L_{\text{target}} = 0.6.$$

Assume the connected eMBB UE counters are

$$K_{2,\text{eMBB}} = 30, \quad K_{3,\text{eMBB}} = 14.$$

The measured eMBB throughput at gNB2 is

$$R_{2,\text{eMBB}} = 3.2 \text{ Mbps}, \quad R_{\min} = 5 \text{ Mbps}.$$

The eMBB constraint is

$$c_{2,\text{eMBB}} = R_{\min} - R_{2,\text{eMBB}} = 5 - 3.2 = 1.8 > 0,$$

so

$$v_{2,\text{eMBB}} = 1.$$

6.2 Global Agent Output

The global state is

$$\mathbf{s}_H(t) = [0.55, 0.60, 0.50, 0.57, 0.92, 0.54, 0.53, 0.45, 0.51, 0, 0, 0, 0, 1, 0, 0, 0, 0].$$

Assume the global PPO agent outputs

$$B(t) = \begin{bmatrix} 0.0 & +0.2 & 0.0 \\ 0.0 & -0.8 & 0.0 \\ 0.0 & +0.3 & 0.0 \end{bmatrix}.$$

The important value is

$$b_{2,\text{eMBB}} = -0.8,$$

which means that gNB2 should offload eMBB users.

6.3 Local Agent 2 Observation

Assume

$$\mathcal{N}_2 = \{gNB_1, gNB_3\}.$$

The global agent has already compressed the load situation into

$$b_{2,\text{eMBB}} = -0.8,$$

meaning that gNB2 should offload eMBB traffic. The local agent does not observe $L_{2,\text{eMBB}}$, $L_{1,\text{eMBB}}$, or $L_{3,\text{eMBB}}$ directly.

For neighbor gNB_1 and slice eMBB, assume

$$K_{1,\text{eMBB}} = 22, \quad \text{Offset}_{2,1,\text{eMBB}}^{\text{prev}} = 0, \quad R_{2,1,\text{eMBB}}^{\text{HF}} = 0.12, \quad R_{2,1,\text{eMBB}}^{\text{PP}} = 0.18.$$

Then

$$\mathbf{o}_{2,1,\text{eMBB}} = [-0.8, 30, 1, 22, 0, 0.12, 0.18].$$

For neighbor gNB_3 , assume

$$K_{3,\text{eMBB}} = 14, \quad \text{Offset}_{2,3,\text{eMBB}}^{\text{prev}} = 0, \quad R_{2,3,\text{eMBB}}^{\text{HF}} = 0.05, \quad R_{2,3,\text{eMBB}}^{\text{PP}} = 0.04.$$

Then

$$\mathbf{o}_{2,3,\text{eMBB}} = [-0.8, 30, 1, 14, 0, 0.05, 0.04].$$

The full state $\mathbf{o}_2(t)$ concatenates this information for all slices and all neighbors. The local agent therefore knows the strategic direction, the SLA status, the UE counters, and the mobility safety of each neighbor, but it does not directly observe the PRB load values.

6.4 Local Actor Output and Quantization

The TD3 actor of local agent 2 outputs continuous proto-offsets. For eMBB, suppose it outputs

$$\widetilde{\text{Offset}}_{2,1,\text{eMBB}} = +3.7, \quad \widetilde{\text{Offset}}_{2,3,\text{eMBB}} = -4.6.$$

Quantization gives

$$\text{Offset}_{2,1,\text{eMBB}}^{\text{applied}} = Q(3.7) = +4,$$

$$\text{Offset}_{2,3,\text{eMBB}}^{\text{applied}} = Q(-4.6) = -4.$$

This means the system discourages handover from gNB2 to gNB1 and encourages handover from gNB2 to gNB3.

6.5 A3 Calculation

For eMBB users served by gNB2, the A3 condition toward gNB3 is

$$\text{RSRP}_3 > \text{RSRP}_2 + \text{Offset}_{2,3,\text{eMBB}}^{\text{applied}}.$$

With $\text{Offset}_{2,3,\text{eMBB}}^{\text{applied}} = -4$, the rule is

$$\text{RSRP}_3 > \text{RSRP}_2 - 4.$$

Consider five eMBB UEs:

UE	RSRP ₂ serving	RSRP ₃ neighbor	RSRP ₃ > RSRP ₂ - 4?
UE1	-82 dBm	-80 dBm	Yes
UE2	-78 dBm	-79 dBm	Yes
UE3	-85 dBm	-84 dBm	Yes
UE4	-75 dBm	-78 dBm	Yes
UE5	-88 dBm	-87 dBm	Yes

All five UEs satisfy the A3 condition for gNB3. Suppose four handovers succeed and one fails:

$$N_{2,3,\text{eMBB}}^{\text{HOA}} = 5, \quad N_{2,3,\text{eMBB}}^{\text{HOS}} = 4, \quad N_{2,3,\text{eMBB}}^{\text{HOF}} = 1.$$

Suppose one ping-pong event occurs:

$$N_{2,3,\text{eMBB}}^{\text{PP}} = 1.$$

Then

$$R_{2,3,\text{eMBB}}^{\text{HF}} = \frac{1}{5} = 0.20, \quad R_{2,3,\text{eMBB}}^{\text{PP}} = \frac{1}{4} = 0.25.$$

6.6 Load and UE Counter Update

If four eMBB UEs successfully move from gNB2 to gNB3 and each contributes 0.03 normalized load, then

$$\begin{aligned} L_{2,\text{eMBB}}^{\text{new}} &= 0.92 - 4(0.03) = 0.80, \\ L_{3,\text{eMBB}}^{\text{new}} &= 0.45 + 4(0.03) = 0.57. \end{aligned}$$

The UE counters become

$$\begin{aligned} K_{2,\text{eMBB}}^{\text{new}} &= 30 - 4 = 26, \\ K_{3,\text{eMBB}}^{\text{new}} &= 14 + 4 = 18. \end{aligned}$$

6.7 Local Reward Calculation

Assume after handover the eMBB throughput becomes

$$R_{2,\text{eMBB}} = 5.6 \text{ Mbps.}$$

Then

$$c_{2,\text{eMBB}} = 5 - 5.6 = -0.6,$$

so

$$\max(0, c_{2,\text{eMBB}}) = 0.$$

Using only the eMBB part for illustration and ignoring the small offset-smoothing term, suppose the tracking penalty for the selected proto-offset toward gNB3 is 0.04. Then

$$\tilde{r}_{L,2} = -0.04 - 1(0.20) - 0.3(0.25).$$

Therefore,

$$\tilde{r}_{L,2} = -0.04 - 0.20 - 0.075 = -0.315.$$

If mobility had been better, for example $R^{\text{HF}} = 0.05$ and $R^{\text{PP}} = 0.04$, then

$$\tilde{r}_{L,2} = -0.04 - 0.05 - 0.3(0.04) = -0.102.$$

The local agent therefore learns that an offset is good only if it follows the global bias while preserving stable handover behavior and SLA satisfaction. The load-balancing success itself is evaluated by the global reward, not by direct local PRB-load observation.

7 Training Workflow

The training process is performed in simulation so the agent is allowed to make poor decisions and learn from their consequences through the reward and Lagrangian penalties. To make learning stable, the environment difficulty is increased progressively. The policy first learns slice-aware load adaptation with fixed UE positions, then adapts to user arrivals and departures, and only later learns under mobility-induced handovers.

7.1 Curriculum Training Scenarios

The recommended training curriculum is:

1. **Stage A - Static UEs with variable demand.** All UEs have velocity $v_u = 0$. UE positions and serving-cell radio conditions are fixed. Only the traffic demand changes over time. This stage isolates the load-balancing and SLA-learning problem from mobility noise. For example, an eMBB traffic multiplier can be applied near gNB2 so that $L_{2,\text{eMBB}}$ increases from a normal value to an overloaded value.
2. **Stage B - Static UEs with appearance and disappearance.** UE positions remain fixed, but UEs can enter or leave the scenario. This changes the connected-UE counters $K_{i,s}(t)$ and the slice loads $L_{i,s}(t)$ without changing radio geometry. This stage teaches the agents to react to changes in the number of active users.
3. **Stage C - Slow controlled mobility.** UEs move with low speed, for example 3–10 km/h. This introduces natural A3 handovers while keeping the mobility dynamics simple enough for debugging the mobility penalties $R_{i,j,s}^{\text{HF}}$ and $R_{i,j,s}^{\text{PP}}$.
4. **Stage D - SUMO mobility.** UE positions and speeds are generated by the SUMO map and route files. This is the final realistic training and evaluation stage. The agent still observes only aggregated KPIs, while UE movement affects RSRP, handover attempts, handover failures, ping-pong events, load, and SLA values through the simulator.

7.2 Stage A: Static UEs with Variable Demand

In the first stage, the objective is to learn the relation between traffic demand, slice load, global bias, and local offset control. Since $v_u = 0$, UE positions are fixed and the radio measurements are stable:

$$\text{RSRP}_i(t) \approx \text{RSRP}_i(0), \quad \text{RSRP}_j(t) \approx \text{RSRP}_j(0).$$

The environment changes only the traffic demand, for example by multiplying the rate of selected eMBB UEs:

$$D_{u,\text{eMBB}}(t) = m(t)D_{u,\text{eMBB}}(0),$$

where $m(t)$ is a demand multiplier. This modifies the load

$$L_{i,s}(t) = \frac{\text{PRB}_{\text{used},i,s}(t)}{\text{PRB}_{\text{total},i,s}(t)}$$

and possibly the SLA constraint $c_{i,s}(t)$, even though the UE positions do not change. This stage is useful for verifying that the global PPO agent learns to output negative bias for overloaded gNB-slice pairs and that the local TD3 agent learns to translate this bias into safe proto-offsets using UE counters and mobility feedback.

7.3 Stage B: Static UEs with Arrival and Departure

In the second stage, UEs may appear and disappear while keeping zero velocity after creation. Let $A_s(t)$ and $D_s(t)$ denote the number of arriving and departing UEs for slice s during a control interval. The connected UE counter is updated as

$$K_{i,s}(t+1) = K_{i,s}(t) + A_{i,s}(t) - D_{i,s}(t) + H_{i,s}^{\text{in}}(t) - H_{i,s}^{\text{out}}(t),$$

where $H_{i,s}^{\text{in}}$ and $H_{i,s}^{\text{out}}$ are successful handovers into and out of gNB i . In this stage, handovers may still occur if the selected A3 offsets make a fixed UE satisfy the A3 condition, but the main source of variation is the user population and traffic demand. This stage tests whether the policy can adapt to sudden increases and decreases in active UEs.

7.4 Stage C: Slow Mobility

After the policy is stable under fixed-position traffic dynamics, slow mobility is enabled. UE movement changes RSRP gradually:

$$\text{RSRP}_i(t+1) \neq \text{RSRP}_i(t), \quad \text{RSRP}_j(t+1) \neq \text{RSRP}_j(t).$$

The gNB continues to apply the standard A3 condition:

$$\text{RSRP}_j(t) > \text{RSRP}_i(t) + \text{Offset}_{i,j,s}^{\text{applied}}(t).$$

This stage validates whether the local reward properly discourages aggressive offsets that create handover failures or ping-pong handovers.

7.5 Stage D: SUMO-Based Mobility

The final stage uses the SUMO map and route files. Each moving UE receives its position and velocity from SUMO. The wireless simulator converts UE positions into radio measurements, evaluates the A3 handover rule, updates serving gNB associations, and reports aggregated KPIs. The agents do not receive UE trajectories directly. The global agent observes load, SLA flags, and optionally connected-UE counters. The local agents observe only the global bias, SLA flags, connected-UE counters, previous proto-offsets, and mobility ratios.

7.6 Complete HRL Training Pipeline

Algorithm 3 Curriculum-Based Complete HRL Training Pipeline

- 1: **Phase 0: Simulator setup.** Configure the radio simulator, SUMO map, gNBs, slices, UE classes, KPI windows, and A3 handover model.
 - 2: **Phase 1: Static demand curriculum.** Train on static UEs with variable demand to learn basic slice load adaptation.
 - 3: **Phase 2: Arrival/departure curriculum.** Keep UEs static after creation, but allow UEs to appear and disappear to learn population adaptation.
 - 4: **Phase 3: Slow mobility curriculum.** Enable low-speed mobility to introduce controlled handovers and mobility penalties.
 - 5: **Phase 4: SUMO mobility curriculum.** Train and evaluate using SUMO routes and realistic movement.
 - 6: **Phase 5: Shared local pretraining.** Train one shared local TD3 policy using rule-based global biases and execute it on all gNBs.
 - 7: **Phase 6: Global PPO training.** Freeze the shared local TD3 policy, deploy it as N local executors, and train the global PPO agent.
 - 8: **Phase 7: Joint fine-tuning.** Unfreeze the shared local TD3 policy and fine-tune both levels with smaller learning rates.
 - 9: **Phase 8: Validation and export.** Validate load balance, SLA satisfaction, handover failure ratio, ping-pong ratio, and export the trained policies.
-

7.7 Training Hyperparameters

Parameter	Global PPO	Local TD3
Learning rate	3×10^{-4}	Actor 10^{-4} , critic 10^{-3}
Discount factor	0.99	0.95
Batch size	64	128
Replay buffer	N/A	10^5 to 10^6
Target noise	N/A	0.2 clipped to 0.5
Policy delay	N/A	2
Multiplier LR η_λ	N/A	10^{-3}

7.8 Why Poor Decisions Are Useful During Training

At the beginning of training, the local actor may choose bad offsets. This is necessary. If the actor chooses an offset that worsens SLA or mobility stability, the Lagrangian penalty and mobility penalty make the reward poor. Load degradation is captured at the global level. The critics then learn that this action has low value in that state, and the actor is updated away from it. Therefore, the training process should let the agent experience the consequence of its decisions inside the simulator.

8 Training Strategy

The proposed training strategy is designed to avoid the instability of learning the global PPO agent and all local TD3 controllers from scratch at the same time. The recommended

configuration is

1 global PPO agent + 7 local TD3 executors with shared parameters.

This means that the simulator contains the full seven-gNB network, but the local level uses one shared TD3 neural policy that is executed independently at each gNB with different local observations. If the network has $N = 7$ gNBs, the global policy outputs

$$\mathbf{b}(t) = [b_{i,s}(t)]_{i=1,\dots,7, s \in \mathcal{S}} \in [-1, 1]^{7S},$$

while each local executor applies the same actor network μ_{θ_L} to its own observation:

$$\tilde{\mathbf{a}}_{L,i}(t) = \mu_{\theta_L}(\mathbf{o}_i(t)), \quad i = 1, \dots, 7.$$

The local policies are therefore decentralized in execution, but parameter-shared in training:

$$\theta_L^{(1)} = \theta_L^{(2)} = \dots = \theta_L^{(7)} = \theta_L.$$

8.1 Why Not Train Only One Lower Agent?

Training only one local agent is easier, but it weakens the HRL contribution because the global outputs for the other gNBs are unused or replaced by rules. It also does not demonstrate network-wide coordination. In contrast, using seven local executors tests the complete seven-gNB system: every global bias has an effect, and the experiment shows coordination between multiple cells and slices.

Training seven independent TD3 agents is possible, but it is more difficult because each local agent changes the environment seen by the other agents. Parameter sharing is the best compromise: the system still has seven local decision makers during execution, while training only one local neural policy.

8.2 Information Split During Training

The two levels learn different responsibilities. The global PPO agent learns the strategic load-balancing decision, while the local TD3 policy learns safe mobility execution.

Aspect	Global PPO Agent	Local TD3 Executor
Question	Which gNB-slice should offload, remain neutral, or retain?	How should the bias be translated into safe A3 offsets?
Input	Network-wide $L_{i,s}$, $v_{i,s}$, optionally $K_{i,s}$	$b_{i,s}$, SLA flag, UE counters, previous offset, HFR, PPR
Output	$b_{i,s}$ for all seven gNBs and all slices	$\widetilde{\text{Offset}}_{i,j,s}$ for local neighbors and slices
Timescale	Slow, T_H	Fast, T_L
Role	Strategic coordination	Tactical mobility control

This information split is important because the local TD3 executors do not observe direct PRB load values. Therefore, they cannot replace the global load-balancing role. The global bias is the compressed instruction that makes the lower level aware of the network-wide decision.

8.3 Phase 1: Pretrain the Shared Local TD3 Policy

In the first phase, the global PPO policy is not trained yet. Instead, a simple rule-based bias is used:

$$b_{i,s} = \begin{cases} -1, & L_{i,s} > 0.75 \quad \text{offload,} \\ +1, & L_{i,s} < 0.45 \quad \text{retain,} \\ 0, & \text{otherwise.} \end{cases}$$

The shared local TD3 policy is trained to convert these rule-based biases into safe slice-specific A3 offsets. The objective of this phase is not to learn global coordination. The objective is to teach the local policy the following behavior:

When I receive an offload or retain instruction, how do I choose neighbor-specific A3 offsets while avoiding handover failures, ping-pong handovers, SLA violations, and unstable offset oscillations?

At the end of this phase, we obtain one generic local controller μ_{θ_L} that can be executed at any gNB.

8.4 Phase 2: Freeze the Local Policy and Train the Global PPO Agent

In the second phase, the trained local policy is frozen and deployed as seven local executors:

$$\mu_{\theta_L}^{(1)}, \mu_{\theta_L}^{(2)}, \dots, \mu_{\theta_L}^{(7)}.$$

All seven executors use the same frozen parameters, but they receive different observations from their corresponding gNBs. The global PPO agent is then trained to output the bias matrix:

$$\pi_H(\mathbf{s}_H(t)) \rightarrow \mathbf{b}(t).$$

Since the local executors are already able to translate bias values into safe A3 offsets, the global agent can focus on learning when each gNB-slice should offload, remain neutral, or retain users. This phase makes the upper-level policy meaningful because all seven lower-level executors react to its decisions.

8.5 Phase 3: Joint Fine-Tuning

In the final phase, the local TD3 policy is unfrozen and both levels are trained together with smaller learning rates. The goal is to adapt the global and local policies to each other without destroying the stable behavior learned in the previous phases. A typical choice is to reduce the learning rates by a factor of 5 to 10 during this phase.

During joint fine-tuning, the global PPO agent continues to update every T_H , while the local TD3 executors update every T_L . The simulator applies all quantized A3 offsets, updates the serving-cell associations, and reports load, SLA, and mobility KPIs. The global reward evaluates network-wide slice-load balance, while the local reward evaluates safe execution of the bias.

8.6 Training Loop Summary

Algorithm 4 Three-Phase Training Strategy with Shared Local TD3 Policy

- 1: Initialize the seven-gNB simulator, slice demands, mobility model, KPI windows, and A3 offset set.
 - 2: **Phase 1: Local pretraining.** Use rule-based biases $b_{i,s}$ and train one shared local TD3 policy μ_{θ_L} across all gNBs.
 - 3: **Phase 2: Global training.** Freeze μ_{θ_L} , deploy it as seven local executors, and train the global PPO policy π_H .
 - 4: **Phase 3: Joint fine-tuning.** Unfreeze μ_{θ_L} and fine-tune π_H and μ_{θ_L} together using smaller learning rates.
 - 5: Validate the final policy using load variance, SLA violation rate, handover failure ratio, ping-pong ratio, and average reward.
-

8.7 Baselines for Evaluation

The final experimental comparison should include the following baselines:

1. **Default fixed A3 offset:** no learning and no slice-specific adaptation.
2. **Rule-based bias + rule-based local offset:** a fully heuristic hierarchy.
3. **Rule-based bias + shared local TD3:** evaluates the local learning contribution alone.
4. **Global PPO + rule-based local offset:** evaluates the global learning contribution alone.
5. **Full HRL:** global PPO with seven shared-weight local TD3 executors.

The main metrics are slice-load variance, SLA violation rate, handover failure ratio, ping-pong ratio, and average reward.

9 Validation During Training

Validation should be performed progressively. The complete HRL system should not be evaluated only after all models are trained. Instead, the local TD3 executor, the global PPO agent, and the full HRL architecture should be validated separately and then together. This prevents credit-assignment confusion and makes it easier to identify whether a performance problem comes from the strategic global policy, the tactical local policy, or the simulator itself.

9.1 Simulator Sanity Validation

Before validating the learning agents, the simulator must be checked using deterministic rules. The purpose is to confirm that the environment reacts correctly to A3 offset changes. In particular, when the applied offset is negative, the A3 condition

$$\text{RSRP}_j(t) > \text{RSRP}_i(t) + \text{Offset}_{i,j,s}^{\text{applied}}(t)$$

should become easier to satisfy, causing more handover attempts toward neighbor j . When the applied offset is positive, the condition should become harder to satisfy, causing fewer handover attempts. This test verifies that offset control changes the effective handover boundary without changing the physical coverage or transmit power.

Useful simulator checks are:

- fixed offset 0 dB should reproduce the default A3 behavior;
- negative offsets should increase handover attempts toward the target neighbor;
- positive offsets should reduce handover attempts toward the target neighbor;
- successful handovers should update $K_{i,s}$, $K_{j,s}$, $L_{i,s}$, and $L_{j,s}$ consistently;
- ping-pong events and handover failures should be counted over a sliding window.

9.2 Validation of the Local TD3 Policy

The local TD3 policy should first be validated without training the global PPO agent. During this phase, the global bias is produced by a simple rule-based policy:

$$b_{i,s} = \begin{cases} -1, & L_{i,s} > 0.75 & \text{offload,} \\ +1, & L_{i,s} < 0.45 & \text{retain,} \\ 0, & \text{otherwise.} \end{cases}$$

The validation question is:

Can the local TD3 policy convert an offload, retain, or neutral instruction into safe neighbor-specific A3 offsets?

The main local validation metric is the tracking error:

$$E_{\text{track},i} = \frac{1}{|\mathcal{S}||\mathcal{N}_i|} \sum_{s \in \mathcal{S}} \sum_{j \in \mathcal{N}_i} \left(\widetilde{\text{Offset}}_{i,j,s} - \psi(b_{i,s}, \kappa_{j,s}) \right)^2.$$

A decreasing $E_{\text{track},i}$ indicates that the local actor is learning to follow the global bias while considering the neighbor-pressure correction. However, low tracking error alone is not sufficient. The local policy must also preserve mobility robustness and SLA satisfaction.

The local TD3 validation metrics are summarized in Table 1.

Metric	Expected behavior during validation
Local reward $\tilde{r}_{L,i}$	Should increase over training.
Tracking error $E_{\text{track},i}$	Should decrease.
Handover failure ratio $R_{i,j,s}^{\text{HF}}$	Should remain low or decrease.
Ping-pong ratio $R_{i,j,s}^{\text{PP}}$	Should remain low or decrease.
SLA violation rate	Should decrease or remain controlled.
Offset oscillation	Should remain limited due to the offset-smoothing penalty.

Table 1: Validation metrics for the local TD3 policy.

A useful local validation scenario is the following. If $b_{2,\text{eMBB}} = -1$, then gNB2 should offload eMBB users. If neighbor gNB3 has low UE pressure and low mobility risk, the local policy should output a negative proto-offset toward gNB3. If neighbor gNB1 has high UE pressure, high handover failure ratio, or high ping-pong ratio, the local policy should avoid aggressive offloading toward gNB1 by choosing a neutral or positive offset. Therefore, the local TD3 policy is valid if it learns to offload only toward safe neighbors.

9.3 Validation of the Global PPO Policy

After the local TD3 policy becomes stable, it is frozen and deployed as N local executors. The global PPO agent is then trained and validated while the lower-level behavior remains fixed. The validation question is:

Can the global PPO agent learn which gNB-slice pairs should offload, retain, or remain neutral?

The main global validation metric is slice-load variance:

$$\text{Var}_L(t) = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} \text{Var}_{i \in \mathcal{G}} (L_{i,s}(t)).$$

A successful global policy should reduce $\text{Var}_L(t)$ and reduce the number of overloaded gNB-slice pairs. The overload ratio can be defined as

$$P_{\text{overload}}(t) = \frac{1}{N|\mathcal{S}|} \sum_{i \in \mathcal{G}} \sum_{s \in \mathcal{S}} \mathbf{1}[L_{i,s}(t) > L_{\max}].$$

The global PPO validation metrics are summarized in Table 2.

Metric	Expected behavior during validation
Global reward r_H	Should increase over training.
Slice-load variance Var_L	Should decrease.
Overload ratio P_{overload}	Should decrease.
Global SLA violation count	Should decrease or remain controlled.
Bias consistency	Overloaded gNB-slice pairs should receive negative bias; lightly loaded pairs should receive positive or neutral bias.

Table 2: Validation metrics for the global PPO policy.

For example, if $L_{2,\text{eMBB}} = 0.92$ and $L_{3,\text{eMBB}} = 0.45$, a reasonable global action is

$$b_{2,\text{eMBB}} < 0,$$

which means that gNB2 should offload eMBB traffic. If gNB3 has enough capacity, the global policy may output a positive or neutral bias for gNB3. The global PPO policy is valid if it consistently learns these strategic load-balancing decisions from the network-wide state.

9.4 Validation of the Complete HRL System

After separate validation of the local and global policies, the complete HRL system is evaluated with both levels active. The validation question is:

Does the hierarchy improve slice-load balance without damaging SLA satisfaction or mobility robustness?

The full HRL model is considered successful when the following trends are observed:

$$\text{Var}_L \downarrow, \quad P_{\text{overload}} \downarrow, \quad \text{SLA violations} \downarrow,$$

while

$$R^{\text{HF}} \text{ remains low}, \quad R^{\text{PP}} \text{ remains low}.$$

The average number of handovers should also be monitored. A policy that improves load balance by creating excessive handovers is not acceptable, because it may increase signaling overhead and mobility instability.

9.5 Validation Episodes Without Exploration Noise

Validation episodes must be separated from training episodes. During training, TD3 uses exploration noise and PPO may sample stochastic actions. During validation, the policies should be evaluated without exploration noise:

$$\tilde{\mathbf{a}}_{L,i} = \mu_{\theta_L}(\mathbf{o}_i),$$

and the global PPO policy should use its deterministic mean action or evaluation-mode output. Validation can be performed every fixed number of episodes, for example every 10 or 20 training episodes.

Validation scenarios should not be identical to training scenarios. The validation set should include unseen traffic demands, unseen UE distributions, different initial loads, different mobility seeds, and, in the final stage, unseen SUMO routes. This checks whether the learned policies generalize beyond the exact situations observed during training.

9.6 Baseline-Based Validation

The proposed HRL method should be validated against the baselines listed in Section 8. The goal is not only to show that the full HRL model improves reward, but also to show which part of the hierarchy contributes to the improvement. A useful comparison is:

1. fixed A3 offset: validates the benefit of adaptive control;
2. rule-based bias with rule-based local offset: validates against a simple heuristic hierarchy;
3. rule-based bias with shared local TD3: isolates the local learning contribution;
4. global PPO with rule-based local offset: isolates the global learning contribution;
5. full HRL: validates the complete global PPO and shared local TD3 design.

The final model should be accepted only if it improves slice-load balance while keeping SLA violation rate, handover failure ratio, ping-pong ratio, and handover count within acceptable limits.

9.7 Practical Validation Checklist

The following checklist can be used during implementation:

Validation step	Question to answer
Simulator validation	Does changing the A3 offset produce the expected handover behavior?
Local TD3 validation	Does the local policy follow the bias safely and avoid risky neighbors?
Global PPO validation	Does the global policy reduce slice-load imbalance?
Full HRL validation	Does the hierarchy improve load balance while preserving SLA and mobility robustness?
Generalization validation	Does the model work on unseen traffic, UE distributions, and mobility routes?
Baseline comparison	Does the full HRL model outperform fixed-offset and rule-based methods?

10 Complexity Analysis

10.1 Global Complexity

The global state dimension is $2NS$ and the global action dimension is NS . With fixed hidden layers, one forward pass scales as

$$\mathcal{O}(NS).$$

For T_H global training steps:

$$\mathcal{C}_{\text{global}} = \mathcal{O}(T_H NS).$$

10.2 Local Complexity

For one local agent, the observation dimension is $7Sd$ and the continuous action dimension is Sd . With fixed hidden sizes, one local forward pass scales as

$$\mathcal{O}(Sd).$$

For N local agents and T_L local training steps:

$$\mathcal{C}_{\text{local}} = \mathcal{O}(T_L NSd).$$

Thus,

$$\boxed{\mathcal{C}_{\text{HRL}} = \mathcal{O}(T_H NS) + \mathcal{O}(T_L NSd)}.$$

A discrete joint-action formulation would require

$$\mathcal{O}(T_L N 7^{Sd}),$$

which is intractable for realistic values of S and d .

11 O-RAN Integration

11.1 Component Mapping

O-RAN nent	Compo- nent	Role
Non-RT RIC rApp		Hosts the global PPO agent and computes biases every 5 seconds.
A1 interface		Sends per-gNB, per-slice biases $b_{i,s}$ to the near-RT RIC.
Near-RT RIC xApp		Hosts one local TD3 agent per gNB and computes proto-offsets every 500 ms.
E2SM-KPM		Reports load and SLA values to the global rApp; reports connected UE counters, SLA values, and mobility counters to local xApps.
E2SM-RC		Sends quantized neighbor-specific slice A3 offsets to the gNB.
O-CU-CP		Applies A3 offset configuration and executes handover control.
O-DU		Provides UE measurements such as RSRP.

11.2 Example KPM Report

```
{
  "node_id": "gNB-42",
  "measurements": [
    {"slice": "URLLC", "load": 0.65, "connected_ues": 4, "delay_ms": 8},
    {"slice": "eMBB", "load": 0.82, "connected_ues": 37, "throughput_mbps": 4.2},
    {"slice": "mMTC", "load": 0.43, "connected_ues": 260, "packet_loss_pct": 0.8}
  ],
  "mobility_counters": [
    {"target_neighbor": "gNB-17", "slice": "eMBB",
      "ho_attempts": 40, "ho_success": 36, "ho_failures": 2, "ping_pong": 1}
  ]
}
```

11.3 Example Control Message

```
{
  "action_type": "A3_OFFSET_CONFIG",
  "target_gNB": "gNB-42",
  "neighbor_slice_configs": [
    {"target_neighbor": "gNB-17", "slice": "eMBB",
      "proto_offset_dB": -4.38, "applied_offset_dB": -4},
    {"target_neighbor": "gNB-19", "slice": "eMBB",
      "proto_offset_dB": 3.21, "applied_offset_dB": 4}
  ]
}
```

```

],
"validity_duration_ms": 500
}

```

12 Deployment Note

In deployment only, a hard safety layer may be added as a final protection mechanism for catastrophic or persistent SLA violations; it is not part of the conceptual formulation or simulator training loop described above.

13 Implementation Notes

- Keep the A3 formula RSRP-based in the main implementation for simplicity.
- Use a generic measurement $M \in \{\text{RSRP}, \text{RSRQ}, \text{SINR}\}$ only as a future extension.
- Store the continuous proto-action in the TD3 replay buffer.
- Execute the quantized offset in the simulator.
- Compute the tracking term using the continuous proto-offset and the bias/UE-pressure target, not direct local PRB load.
- Compute mobility KPIs over sliding windows to reduce noise.
- Local TD3 weights may be shared across gNBs if scalability is important, provided the observation includes enough gNB-specific context.

14 Conclusion

This Version 9.0 formulation keeps the two-agent HRL architecture clear and makes the global agent non-redundant through information asymmetry. The global PPO agent is the component with network-wide slice-load visibility and coordinates demand by producing per-gNB, per-slice biases. The local TD3 agents, one per gNB, do not observe direct PRB load values; they convert the global bias, SLA indicators, UE counters, previous offsets, and mobility counters into continuous neighbor-specific and slice-specific proto-offsets. The proto-offsets are quantized to valid A3 offsets before application. The gNB continues to execute the standard A3 handover procedure. The solution is scalable because the global level grows linearly with NS , and the local level grows linearly with NSd instead of exponentially with 7^{Sd} . In the recommended training strategy, one shared local TD3 policy is executed as seven local controllers, which preserves decentralized behavior while reducing training instability and parameter cost.

A Acronyms

3GPP	3rd Generation Partnership Project
A1	Interface between Non-RT RIC and near-RT RIC
A3	Handover event type
CMDP	Constrained Markov Decision Process
E2	Interface between near-RT RIC and O-DU/O-CU
eMBB	enhanced Mobile Broadband
gNB	Next Generation Node B
HRL	Hierarchical Reinforcement Learning
mMTC	massive Machine Type Communications
Non-RT RIC	Non-Real-Time RAN Intelligent Controller
near-RT RIC	Near-Real-Time RAN Intelligent Controller
PPO	Proximal Policy Optimization
PRB	Physical Resource Block
RSRP	Reference Signal Received Power
SLA	Service-Level Agreement
TD3	Twin Delayed Deep Deterministic Policy Gradient
URLLC	Ultra-Reliable Low-Latency Communications
xApp	RIC Application for near-RT RIC

References

- [1] 3GPP TS 38.331, “Radio Resource Control (RRC) protocol specification,” 2022.
- [2] O-RAN Alliance, “O-RAN Architecture Description,” Technical Specification, 2023.
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” arXiv:1707.06347, 2017.
- [4] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing Function Approximation Error in Actor-Critic Methods,” ICML, 2018.
- [5] E. Altman, “Constrained Markov Decision Processes,” Chapman & Hall/CRC, 1999.
- [6] Q. Liao, T. Hu, and D. Wellington, “Knowledge Transfer in Deep Reinforcement Learning for Slice-Aware Mobility Robustness Optimization,” arXiv:2203.03227, 2022.